Experiment No: 1

Title: Data Preprocessing: Normalisation, Discretisation, Missing Value Handling and Feature Engineering

Batch: C3 **Roll No: 16014225801** **Experiment No.: 01**

Aim: Data Preprocessing: Normalisation, Discretisation, Missing Value Handling and Feature Engineering

---

Resources Used: Any RDBMS, Java, Python

Theory:

Data processing techniques, when applied before mining, can substantially improve the overall quality of the patterns mined and/or the time required for the actual mining. Different kinds of pre-processing tasks are performed on the data before applying mining techniques. Data reduction can reduce data size by, for instance, aggregating, eliminating redundant features, or clustering. Data transformations (e.g., normalization) may be applied, where data are scaled to fall within a smaller range like 0.0 to 1.0. This can improve the accuracy and efficiency of mining algorithms involving distance measurements. These techniques are not mutually exclusive; they may work together. Normalization, data discretization, and concept hierarchy generation are forms of data transformation.

Data reduction techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data. That is, mining on the reduced data set should be more efficient yet produce the same (or almost the same) analytical results. Data reduction strategies include dimensionality reduction, numerosity reduction, and data compression. Dimensionality reduction is the process of reducing the number of random variables or attributes under consideration. Dimensionality reduction methods include wavelet transforms and principal components analysis, which transform or project the original data onto a smaller space. Attribute subset selection is a method of dimensionality reduction in which irrelevant, weakly relevant, or redundant attributes or dimensions are detected and removed. Numerosity reduction techniques replace the original data volume by alternative, smaller forms of data representation. These techniques may be parametric or nonparametric. For parametric methods, a model is used to estimate the data, so that typically only the data parameters need to be stored, instead of the actual data. (Outliers may also be stored.) Regression and log-linear models are examples. Nonparametric methods for storing reduced representations of the data include histograms, clustering, sampling, and data cube aggregation.

Data Normalization scales numerical values into a common range so that features with larger values do not dominate smaller ones. Common normalization techniques include:

1. Min-Max Normalization
2. Z-score Standardization
3. Decimal Scaling

Data Discretization: It is the process of converting continuous numerical values into a finite number of intervals or categories. This technique simplifies data representation and is particularly useful for classification algorithms and rule-based systems. Common

discretization methods include equal-width binning, equal-frequency binning, and entropy-based discretization. For example, age values can be grouped into categories such as Young, Adult, Middle-aged, and Senior Citizen.

Feature Engineering: It refers to the process of creating new features or modifying existing ones to improve the predictive performance of machine learning models. It involves extracting meaningful information from raw data by combining attributes, transforming variables, encoding categorical data, or generating domain-specific features. Examples include calculating Body Mass Index (BMI) from height and weight, extracting the year and month from date fields, or creating interaction features from multiple variables. Effective feature engineering often leads to significant improvements in model accuracy and interpretability.

Procedure / Approach /Algorithm / Activity Diagram:

1. Load a suitable dataset with numerical and categorical attributes.
2. Identify and handle missing values using appropriate imputation strategies.
3. Apply normalization techniques (Min-Max Scaling / Z-score normalization).
4. Discretize numerical features using binning methods (equal-width/equal-frequency).
5. Perform feature engineering including encoding of categorical variables and creation of new features.
6. Visualize and interpret the pre-processed data.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder

# 1. Load Dataset
df = pd.read_csv("bombay2.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

# Separate Numerical & Categorical Columns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object', 'category']).columns

# 2. Handle Missing Values
if len(num_cols) > 0:
    num_imputer = SimpleImputer(strategy='mean')
    df[num_cols] = num_imputer.fit_transform(df[num_cols])
```

```
# Categorical -> Most Frequent
if len(cat_cols) > 0:
    cat_imputer = SimpleImputer(strategy='most_frequent')
    df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

print("\nMissing Values After Imputation:")
print(df.isnull().sum())

# 3. Normalization
# Min-Max Scaling
df_minmax = df.copy()

if len(num_cols) > 0:
    scaler = MinMaxScaler()
    df_minmax[num_cols] = scaler.fit_transform(df[num_cols])

print("\nMin-Max Normalized Data:")
print(df_minmax.head())

# Z-Score Normalization
df_zscore = df.copy()

if len(num_cols) > 0:
    scaler = StandardScaler()
    df_zscore[num_cols] = scaler.fit_transform(df[num_cols])

print("\nZ-Score Normalized Data:")
print(df_zscore.head())

# 4. Discretization (Binning)
df_binned = df.copy()

if len(num_cols) > 0:
    column = num_cols[0]       # First numerical column

    # Equal Width Binning
    df_binned[column + "_EqualWidth"] = pd.cut(
        df[column],
        bins=4,
        labels=["Low", "Medium", "High", "Very High"]
    )

    # Equal Frequency Binning
    df_binned[column + "_EqualFreq"] = pd.qcut(
        df[column],
        q=4,
        labels=["Q1", "Q2", "Q3", "Q4"]
    )

print("\nData after Binning:")
print(df_binned.head())

# 5. Feature Engineering
df_feature = df.copy()

# Encode categorical variables
```

```
for col in cat_cols:
    encoder = LabelEncoder()
    df_feature[col] = encoder.fit_transform(df_feature[col])

# Create a new feature
if len(num_cols) >= 2:
    df_feature["Feature_Sum"] = df[num_cols[0]] + df[num_cols[1]]

print("\nFeature Engineered Data:")
print(df_feature.head())
```

Output:

```
First 5 Rows:
  Region Country             City  Month  Day  Year  Temp
0   Asia   India  Bombay (Mumbai)      1    1  1995  71.8
1   Asia   India  Bombay (Mumbai)      1    2  1995  72.0
2   Asia   India  Bombay (Mumbai)      1    3  1995  70.3
3   Asia   India  Bombay (Mumbai)      1    4  1995   NaN
4   Asia   India  Bombay (Mumbai)      1    5  1995  71.3
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 9266 entries, 0 to 9265
Data columns (total 7 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Region   9266 non-null   str
 1   Country  9266 non-null   str
 2   City     9266 non-null   str
 3   Month    9266 non-null   int64
 4   Day      9266 non-null   int64
 5   Year     9266 non-null   int64
 6   Temp     9066 non-null   float64
dtypes: float64(1), int64(3), str(3)
memory usage: 506.9 KB
None

Missing Values:
Region       0
Country      0
City         0
Month        0
Day          0
Year         0
Temp       200
dtype: int64

Missing Values After Imputation:
Region     0
Country    0
City       0
Month      0
Day        0
Year       0
Temp       0
dtype: int64
```

```
Min-Max Normalized Data:
  Region Country             City  Month       Day  Year      Temp
0   Asia   India  Bombay (Mumbai)    0.0  0.000000   0.0  0.891441
1   Asia   India  Bombay (Mumbai)    0.0  0.033333   0.0  0.892484
2   Asia   India  Bombay (Mumbai)    0.0  0.066667   0.0  0.883612
3   Asia   India  Bombay (Mumbai)    0.0  0.100000   0.0  0.942507
4   Asia   India  Bombay (Mumbai)    0.0  0.133333   0.0  0.888831

Z-Score Normalized Data:
  Region Country             City    Month       Day      Year          Temp
0   Asia   India  Bombay (Mumbai) -1.58192 -1.672229 -1.663929 -9.335185e-01
1   Asia   India  Bombay (Mumbai) -1.58192 -1.558606 -1.663929 -9.144365e-01
2   Asia   India  Bombay (Mumbai) -1.58192 -1.444983 -1.663929 -1.076633e+00
3   Asia   India  Bombay (Mumbai) -1.58192 -1.331360 -1.663929 -1.355856e-15
4   Asia   India  Bombay (Mumbai) -1.58192 -1.217737 -1.663929 -9.812235e-01

Data after Binning:
  Region Country             City  Month  Day    Year       Temp  \
0   Asia   India  Bombay (Mumbai)    1.0  1.0  1995.0  71.800000
1   Asia   India  Bombay (Mumbai)    1.0  2.0  1995.0  72.000000
2   Asia   India  Bombay (Mumbai)    1.0  3.0  1995.0  70.300000
3   Asia   India  Bombay (Mumbai)    1.0  4.0  1995.0  81.584293
4   Asia   India  Bombay (Mumbai)    1.0  5.0  1995.0  71.300000

  Month_EqualWidth Month_EqualFreq
0              Low              Q1
1              Low              Q1
2              Low              Q1
3              Low              Q1
4              Low              Q1

Feature Engineered Data:
   Region  Country  City  Month  Day    Year       Temp  Feature_Sum
0       0        0     0    1.0  1.0  1995.0  71.800000          2.0
1       0        0     0    1.0  2.0  1995.0  72.000000          3.0
2       0        0     0    1.0  3.0  1995.0  70.300000          4.0
3       0        0     0    1.0  4.0  1995.0  81.584293          5.0
4       0        0     0    1.0  5.0  1995.0  71.300000          6.0
```

Questions Answers:

1. Explain with example Min-Max normalization technique.

Min-Max normalization is a scaling technique that transforms numerical values into a fixed range, usually between 0 and 1. It uses the formula: **(X − Min) / (Max − Min)**. For example, if the values are 20, 40, 60, 80, and 100, the normalized value of 60 is **(60−20)/(100−20) = 40/80 = 0.5**. This method preserves the relationships between data points and is commonly used before training machine learning models.

2. Explain with an example the difference between normalization and standardization.

Normalization scales data to a fixed range, typically 0 to 1, while standardization transforms data so it has a mean of 0 and a standard deviation of 1. For example, for values 10, 20, and 30, normalization converts them to 0, 0.5, and 1. Standardization instead produces values based on their distance from the mean. Normalization is useful when feature ranges differ

greatly, whereas standardization is preferred for algorithms that assume normally distributed data.

3. Why is feature engineering important in machine learning?
Feature engineering is important because it improves the quality of input data, enabling machine learning models to achieve better accuracy and performance. It involves creating new features, transforming existing ones, and encoding categorical variables into numerical form. Good feature engineering helps reveal hidden patterns, reduces noise, and simplifies complex relationships within the data. As a result, models learn more effectively, train faster, and often generalize better to unseen data.

4. What are the pros and cons of discretization?
Discretization converts continuous numerical values into discrete intervals or categories. Its advantages include simplifying data, reducing the effect of noise and outliers, and making results easier to interpret. It can also improve the performance of some classification algorithms. However, discretization may lead to information loss because exact numerical values are replaced with ranges. Poorly chosen intervals can reduce model accuracy and fail to capture important patterns in the data

---

Conclusion:
Data preprocessing improves data quality by handling missing values, normalizing and discretizing numerical data, and engineering meaningful features. These techniques enhance model accuracy, efficiency, reliability, and overall machine learning performance.

References:
1. Stephen Marsland, “Machine learning An Algorithmic Perspective CRC”, Press, 2015
2. Christopher M. Bishop “Pattern Recognition and Machine Learning Springer Science, Business Media 2006
3. Simon Haykin, “Neural Networks and Learning Machines” Pearson Education, 2009